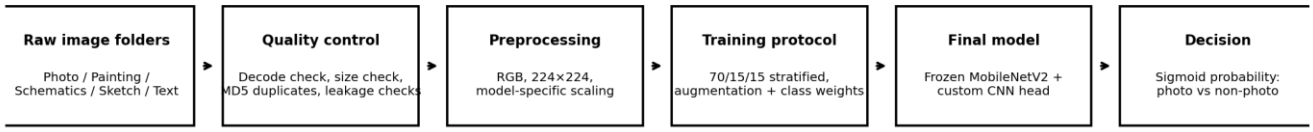
Exploratory Modeling Report

Photo vs Non-Photo Image Classification

Experimental paths, preprocessing choices, model comparisons, and final retained configuration

Retained end-to-end pipeline



Executive Summary

This project investigated how to identify whether an input image is a photographic image or a non-photographic document. The exploration was intentionally broad: the team tested binary and five-class formulations, multiple preprocessing pipelines, two train/validation/test split strategies, and several neural-network families ranging from a simple multilayer perceptron (MLP) to convolutional neural networks (CNNs) and transfer-learning architectures.

The main lesson from the experimentation is that the formulation of the problem mattered as much as the architecture. Multiclass classification provided useful diagnostic information because it exposed the exact source of errors (painting, sketch, schematics, or text), but it made the learning task harder than the actual deliverable required. The binary photo/non-photo formulation was ultimately better aligned with the objective and produced the strongest final metrics.

The retained split was 70% training, 15% validation, and 15% test. An 80/10/10 strategy was also explored during the project, but the team observed better results with 70/15/15.

A subsequent diagnostic cycle on 9 October revisited the five-class models after substantial run-to-run variation was observed in the MLP and scratch CNN. The follow-up work linked the instability to learning-rate choice, oversized dense layers, overly aggressive augmentation, insufficient normalization, and early stopping that was too impatient. **The first completed retraining result was the revised MLP, whose best validation accuracy rose to 74.3%; the remaining revised models were still being retrained at the time of that diagnostic report.**

Final retained combination

Binary classification + quality-controlled data + stratified 70/15/15 split + data augmentation + class weighting + frozen MobileNetV2 feature extractor + custom CNN head + EarlyStopping/ModelCheckpoint. In the recorded binary test evaluation, the hybrid model reached 98.38% accuracy and a 96.88% F1-score for the photo class, the best combination among the final models evaluated in that run.

Final binary model	Test accuracy	Photo precision	Photo recall	Photo F1	Parameters	Model size
Hybrid MobileNetV2	98.38%	96.31%	97.47%	96.88%	3.01 M	18.7 MB

Report Structure

1. Project objective and experimental strategy
2. Dataset understanding and quality control
3. Preprocessing paths explored
4. Data-splitting strategies
5. Multiclass classification exploration
6. Binary classification exploration
7. Architecture and hyperparameter exploration
8. Training controls and bias/variance analysis
9. Comparative results and final selection

1. Project Objective and Experimental Strategy

The practical objective is to answer a binary question: “Is this image a photo?” The available dataset, however, is naturally organized into five folders: Photo, Painting, Schematics, Sketch, and Text. This structure encouraged two complementary problem formulations.

Path	Target	Output layer / loss	Purpose
Multiclass	Predict one of five categories	5-way softmax + sparse categorical cross-entropy	Understand category-level confusions and test whether a richer formulation improves representation learning.
Binary	Photo vs non-photo	1 sigmoid + binary cross-entropy	Match the final business objective directly and optimize photo detection.

The project followed an iterative experimental process rather than a single linear training run. Each iteration answered a specific question: Is the data trustworthy? Is the split leakage-free? Is a generic MLP sufficient? Does a CNN trained from scratch learn the relevant visual patterns? Does transfer learning provide stronger features? Does a custom head on top of MobileNetV2 improve the photo/non-photo decision?

This report therefore focuses on the paths explored and the evidence used to converge on the final configuration, not only on the final score.

2. Dataset Understanding and Quality Control

2.1 Initial data profiling

The dataset contain 41,399 images across the five categories. The raw category sizes were highly uneven because Sketch contained only 1,406 images while the other major categories were close to 10,000 images each. This immediately highlighted class-imbalance risk for multiclass learning and, in the binary formulation, an imbalance between Photo and the union of all non-photo classes.

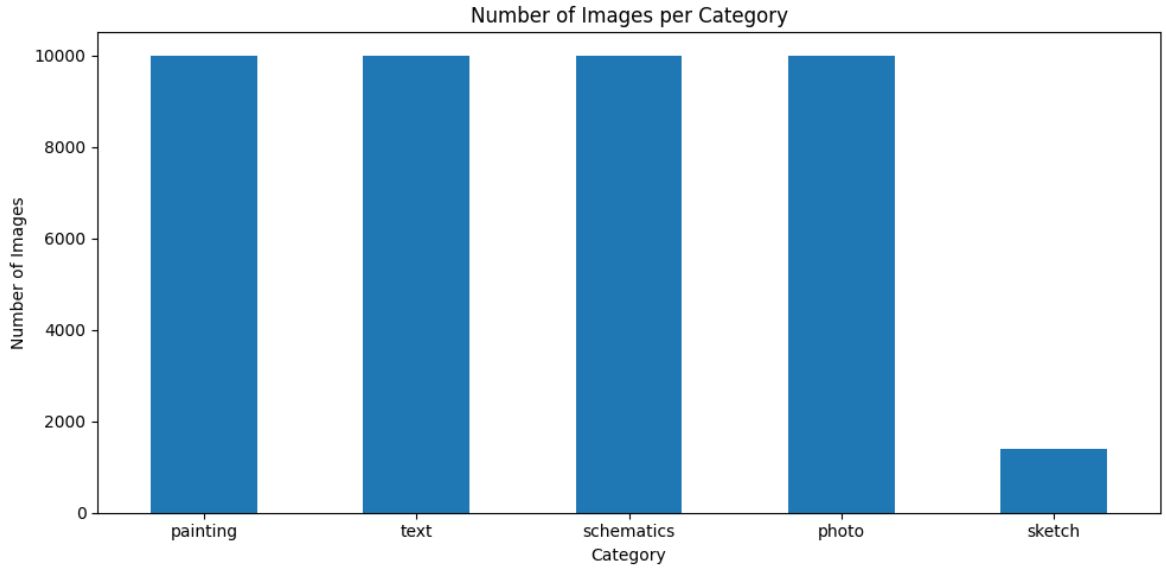


Figure 1. Initial number of images per category before the stronger cleaning pipeline.

The raw image characteristics were also heterogeneous. Images appeared in JPEG, PNG, MPO, GIF, and BMP formats, and color modes included RGB, grayscale (L), palette (P), RGBA, and CMYK. Dimensions varied from extremely small values to images above 10,000 pixels in one dimension. Aspect-ratio outliers were especially visible in the schematics category. These observations motivated explicit image validation and a standardized model input size.

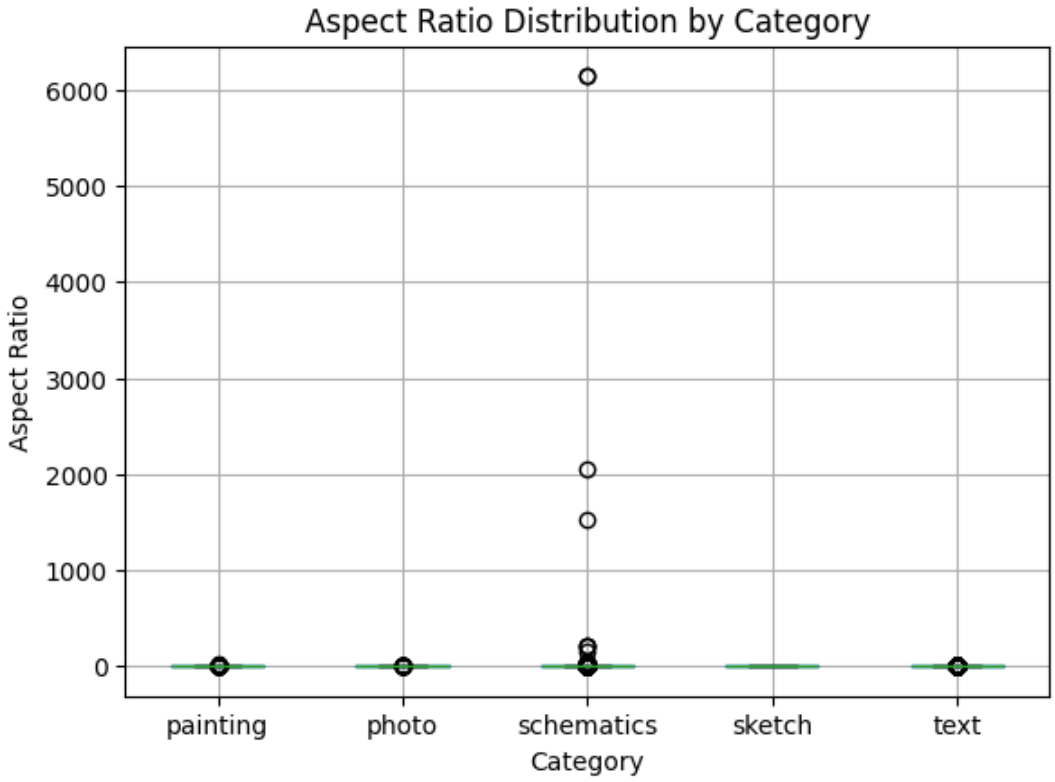


Figure 2. Aspect-ratio distribution by category, showing extreme outliers in schematics.

2.2 Evolution of the data-quality pipeline

The data-quality procedure became stricter over time. The earliest pass used image opening/verification and dimension inspection. Later, the quality check with the actual TensorFlow training decoder and added content fingerprints (MD5), minimum-size rules, and leakage checks. This is a significant methodological improvement: a file that passes the same decoder used in training is far less likely to crash an expensive training run later.

- Readability check using the TensorFlow decoder used by the training pipeline.
- Width/height and color-mode inspection.
- Minimum-side threshold of 32 pixels to remove tiny icons/spacers that are not meaningful documents.
- MD5 content fingerprints to identify exact duplicates even when filenames differ.
- Cross-category duplicate check to detect potential label conflicts.
- Removal of unreadable, too-small, and duplicate samples before splitting.
- Post-split leakage check ensuring the same file/content cannot occur in multiple subsets.

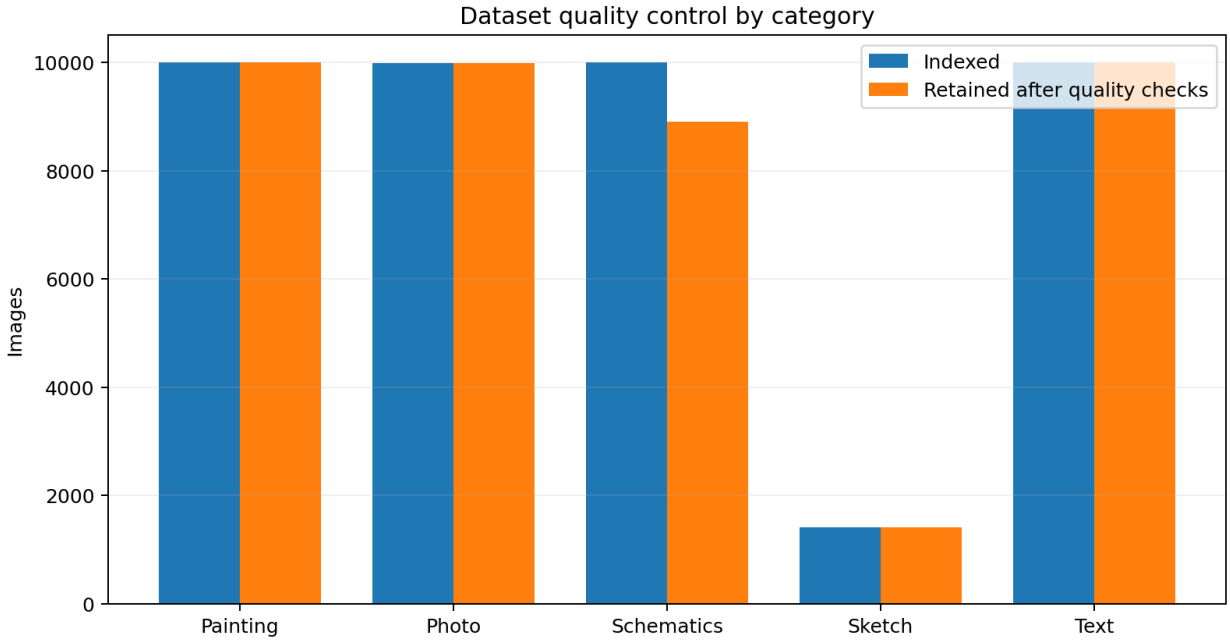


Figure 3. Indexed images versus images retained after quality control in the multiclass experiment snapshot.

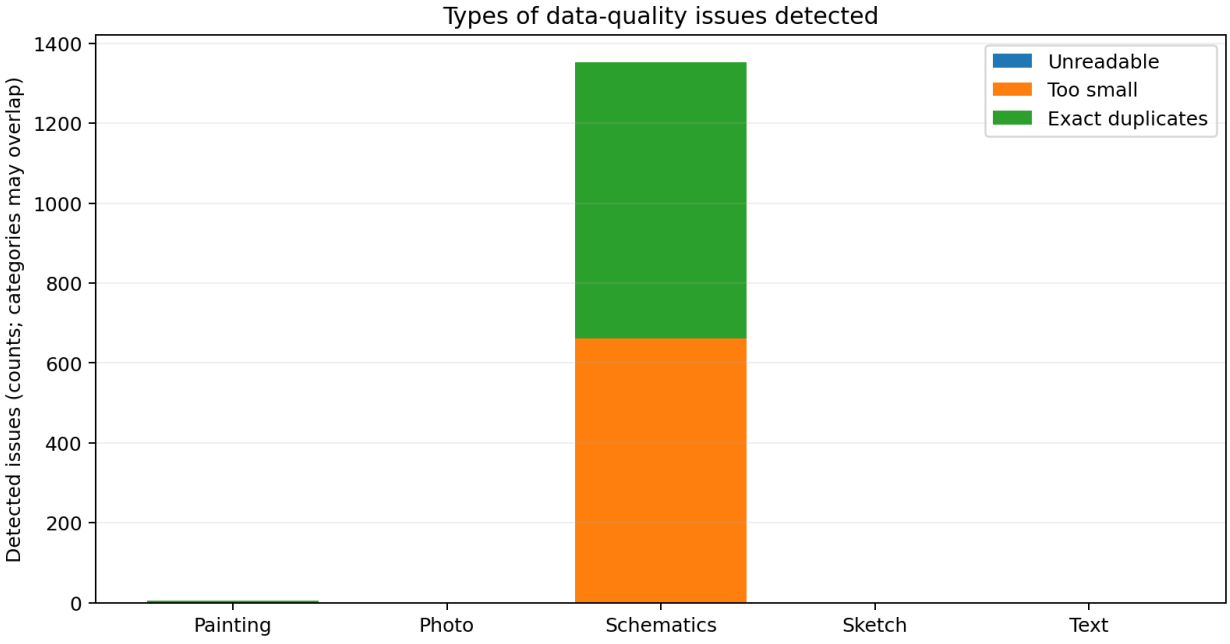


Figure 4. Main quality issues detected. Counts are diagnostic and can overlap for the same file.

In the multiclass run, the notebook recorded one unreadable painting, 662 schematics below the minimum size, and hundreds of exact duplicates (especially in schematics). No identical image was found under two different categories. The cleaned multiclass snapshot retained 40,301 images.

3. Preprocessing Paths Explored

Preprocessing was not fixed from the beginning. Several choices were explored and refined as the team moved from basic experiments toward a more reproducible deployment-oriented pipeline.

Preprocessing element	Explored alternatives	Reason / observation
Image decoding	PIL verification → TensorFlow decoder validation	The later approach matches the actual training decoder and catches runtime-incompatible images earlier.
Color handling	Native modes inspected; model input forced to 3 RGB channels	Provides a consistent tensor shape for grayscale, RGBA, palette, and other input modes.
Resizing	224×224 with padding/aspect-ratio preservation → direct 224×224 resize	Both standardize shape. The later pipeline prioritized a simple, model-integrated path compatible with MobileNetV2.
Pixel scaling	Normalize to [0,1] in the input pipeline → keep [0,255] until model-specific preprocessing	Moving scaling inside the model reduces train/serve mismatch and lets MobileNetV2 use its expected preprocess_input transformation.
Augmentation	None in initial baseline thinking → horizontal flip, rotation 0.1, zoom 0.1	Reduces memorization and improves robustness to small appearance changes.
Imbalance handling	Raw class frequencies → balanced class weights	Avoids discarding data and increases the cost of mistakes on minority classes.
Leakage prevention	Path-overlap check → content-hash leakage check	MD5-based checks are stronger because renamed copies are still detected.



Figure 5. Example of original image versus standardized 224×224 preprocessing.

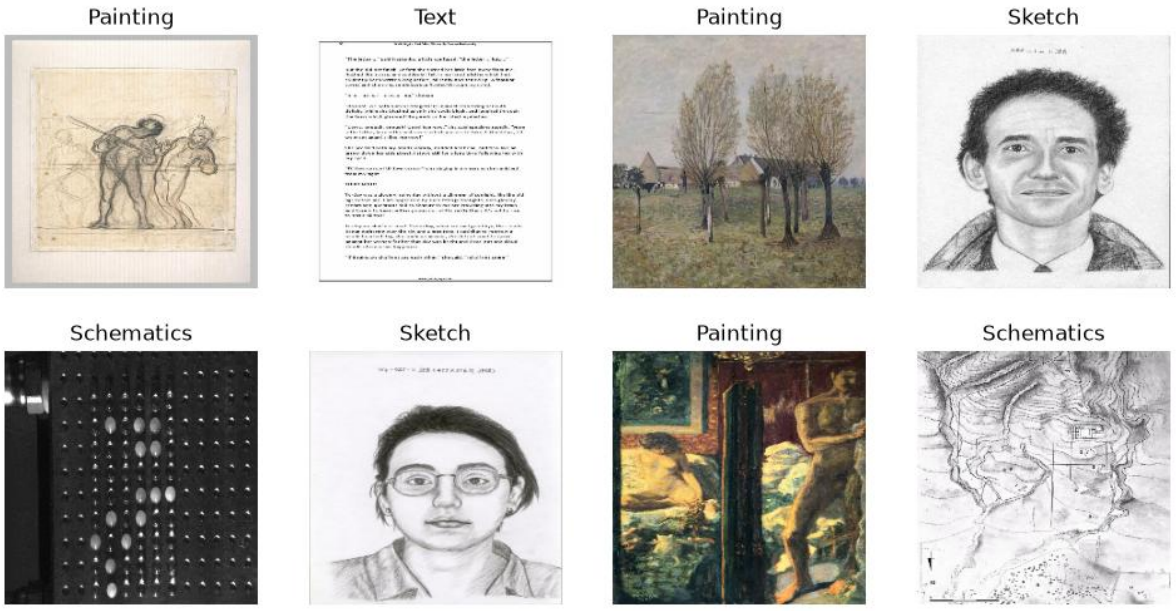


Figure 6. Visual inspection of a training batch used to verify labels and loading behavior before training.

A particularly useful design change was keeping raw pixels in the data pipeline and performing normalization inside the model. The CNN uses Rescaling(1/255), whereas MobileNetV2 uses its dedicated preprocess_input function. This makes preprocessing part of the saved model graph and lowers the risk that production inference applies a different transformation from training.

4. Data-Splitting Strategies

Two split strategies were explored during the project: 70/15/15 and 80/10/10 for train/validation/test. The team retained 70/15/15 because it gave better observed results.

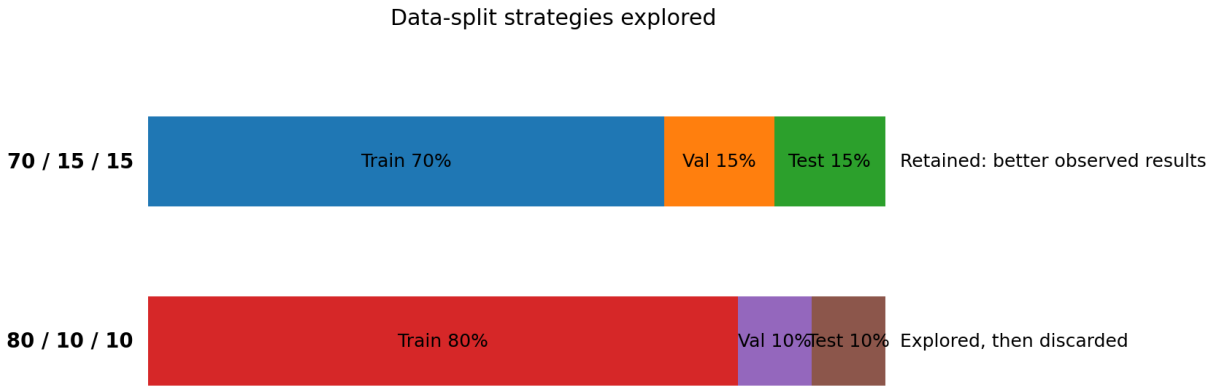


Figure 7. Split strategies explored and the retained choice.

The retained split was stratified by the original five-category label rather than only by the binary photo/non-photo label. This is a strong choice because it preserves the relative presence of Painting, Schematics, Sketch, and Text in all subsets. Since the binary label is derived from the category, binary balance is preserved as well.

Recorded run	Clean images	Train	Validation	Test	Stratification
Multiclass snapshot	40,301	28,210 (70%)	6,045 (~15%)	6,046 (~15%)	Original category
Binary snapshot	38,718	27,102 (70%)	5,808 (15%)	5,808 (15%)	Original category

Why the 70/15/15 split can be advantageous here: the validation set is large enough to give a more stable signal for EarlyStopping and architecture comparison, while the test set remains large enough for reliable final estimates. An 80/10/10 split offers more training data, but smaller validation/test samples can make the selection signal noisier. In this project the empirical comparison favored 70/15/15.

5. Multiclass Classification Exploration

The multiclass experiment asked the model to predict one of five categories. This was useful even though the final deliverable is binary: the confusion matrix can reveal whether photos are being confused specifically with paintings, schematics, sketches, or text.

5.1 MLP baseline

The MLP was deliberately used as a weak image baseline. Flattening a 224×224×3 image destroys spatial locality, so the network must independently learn patterns for many pixel positions. Two MLP configurations appear across the project. An earlier binary baseline used Dense layers of 512→256→256 with dropout 0.4 and approximately 77.3 million trainable parameters. The multiclass revision used a much smaller 128→64 configuration with dropout 0.5 and BatchNormalization after the first dense layer.

There is an important failed variant: without BatchNormalization, the first multiclass MLP became numerically unstable (loss around 40 after only a few batches) and collapsed toward the same prediction for every image. Adding BatchNormalization stabilized training. This is a valuable exploratory result because it shows how normalization can matter even before comparing architectures.

5.2 CNN from scratch

The CNN introduced spatial inductive bias: two convolution/pooling blocks (32 and 64 filters), dropout 0.25 after each block, then a Dense(64) decision layer with dropout 0.5. Compared with the MLP, it uses fewer parameters and can recognize local visual patterns such as edges, textures, and shapes regardless of their exact position.

5.3 Pure transfer learning

The transfer-learning path used MobileNetV2 pre-trained on ImageNet as a frozen feature extractor. Its 7×7×1280 feature map was reduced by GlobalAveragePooling2D, followed by dropout 0.2 and a trainable output layer. Because only a small head is learned from the project data, this approach converged quickly and produced a large jump in generalization performance.

5.4 Hybrid transfer learning + custom CNN

The hybrid path preserved the frozen MobileNetV2 backbone but replaced the simple global-average-pooling head with a small custom CNN: Conv2D(64) → MaxPooling2D → Flatten → Dense(32) → output. This allowed the project team to learn task-specific combinations of the pretrained feature grid while keeping the expensive backbone frozen.

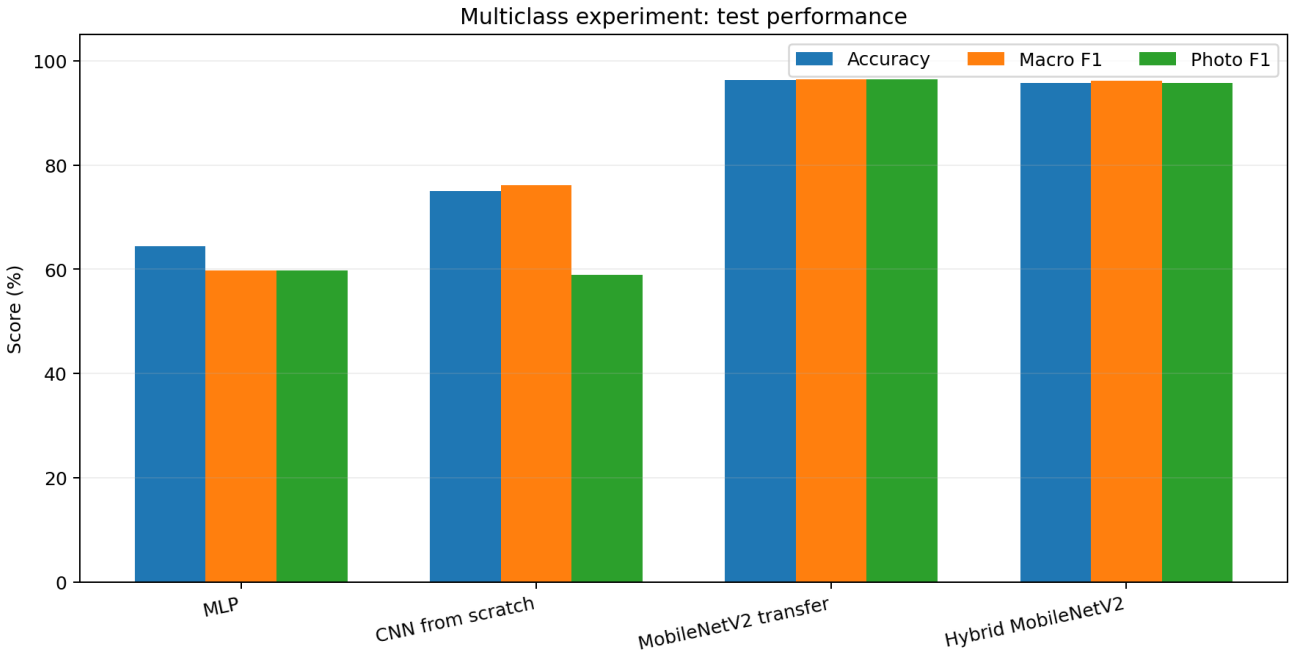


Figure 8. Multiclass test comparison: MLP and scratch CNN versus transfer-learning approaches.

Model	Accuracy	Macro F1	Photo precision	Photo recall	Photo F1
MLP baseline	64.44%	59.74%	62.19%	57.51%	59.76%
CNN from scratch	75.06%	76.14%	59.47%	58.24%	58.85%
MobileNetV2 transfer	96.31%	96.51%	94.95%	97.93%	96.42%
Hybrid MobileNetV2	95.75%	96.10%	93.97%	97.67%	95.78%

In the five-class task, pure MobileNetV2 transfer learning achieved the best overall test accuracy (96.31%) and macro F1 (96.51%), slightly above the hybrid model. Both transfer-learning models dramatically outperformed the MLP and the CNN trained from scratch.

The category-level confusion analysis also revealed the main semantic difficulty: paintings were the category most likely to be confused with photos. For the pure transfer model, 4.1% of paintings were predicted as photos, while photo recall reached 97.9%. This helped justify later focusing the optimization directly on the binary photo/non-photo boundary.

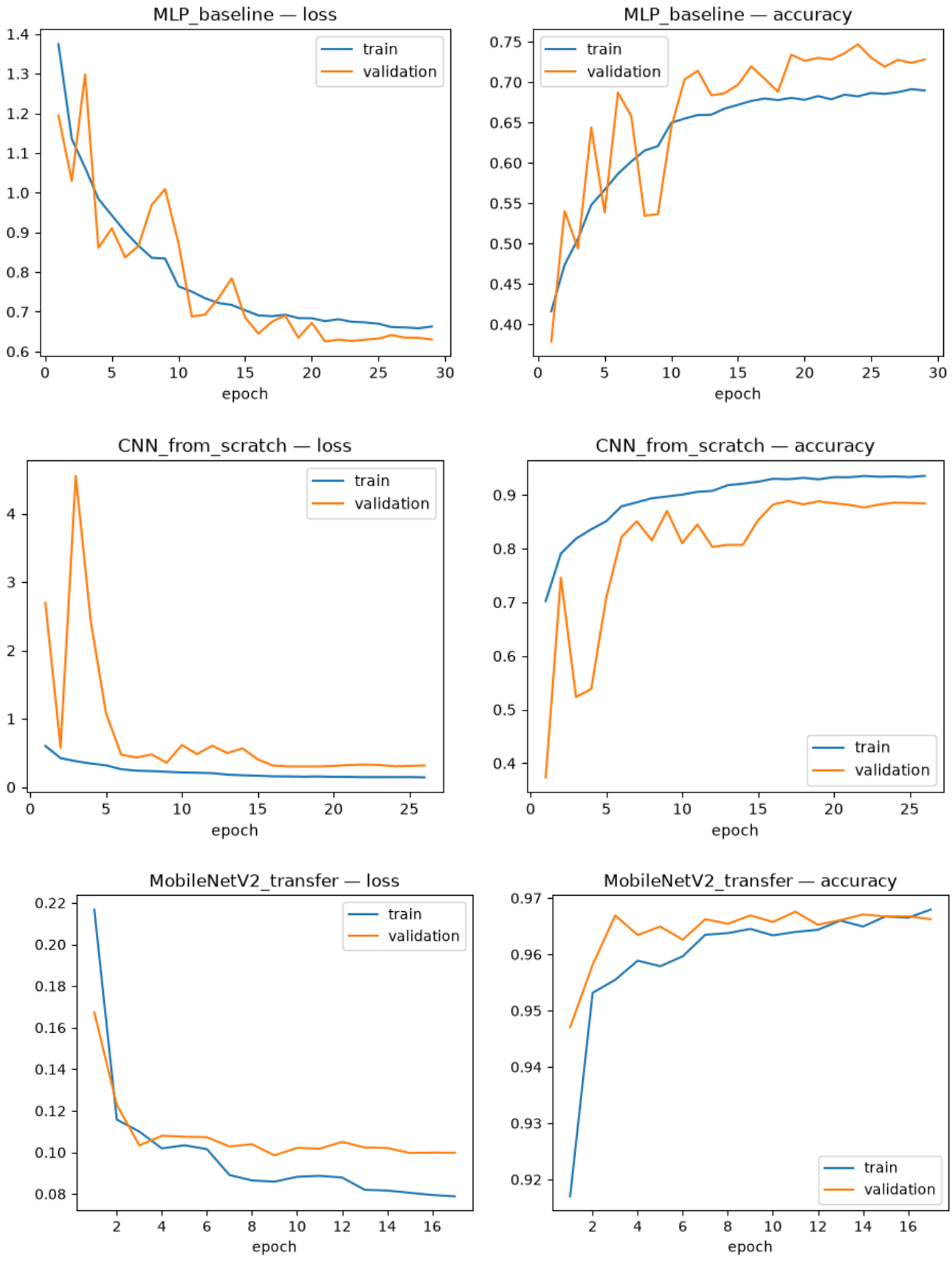


Figure 9. Multiclass MobileNetV2 transfer-learning curves.

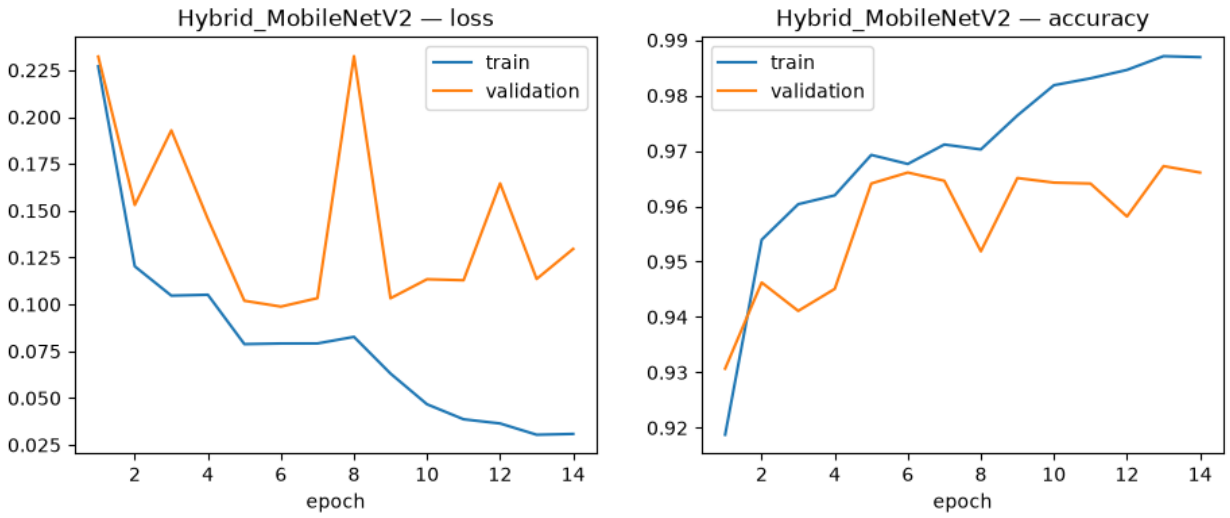


Figure 10. Multiclass hybrid MobileNetV2 curves.

6. Binary Classification Exploration

The project then returned to the direct objective: Photo = 1 and Painting/Sketch/Schematics/Text = 0. This simplifies the output space and lets the loss function focus only on the decision boundary that matters for the deliverable.

In the initial raw profile, the binary distribution was strongly imbalanced: approximately 24% Photo and 76% Non-photo. Rather than undersampling the majority class or duplicating photos, the experiments used balanced class weights computed from the training set. This preserves all available data while making errors on the minority photo class more costly.

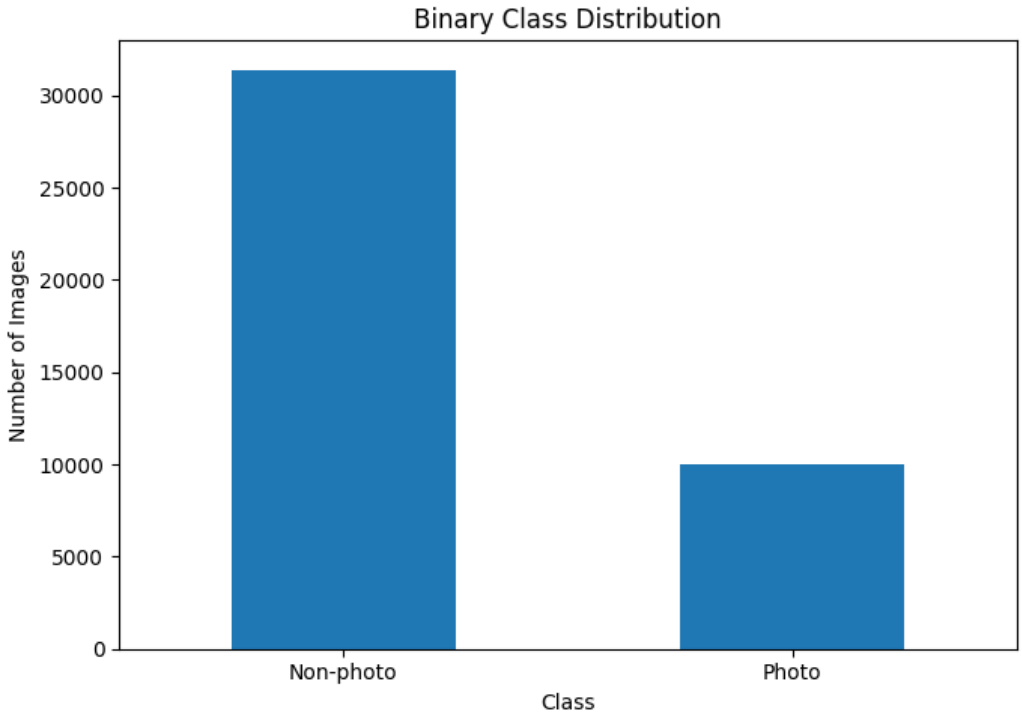


Figure 11. Initial binary class distribution, motivating class-weighted training.

6.1 Binary architecture comparison

The final binary notebook compared three models under the same data split and training protocol: a CNN from scratch, pure MobileNetV2 transfer learning, and the hybrid MobileNetV2 model. The output layer became a single sigmoid neuron, and binary cross-entropy replaced the multiclass loss.



Figure 12. Binary test results for the three final architectures.

Model	Accuracy	Photo precision	Photo recall	Photo F1	ms/image	Parameters	Size
CNN from scratch	87.84%	72.80%	84.46%	78.20%	6.28	11.96 M	143.6 MB
MobileNetV2 transfer	98.02%	96.13%	96.20%	96.17%	11.00	2.26 M	9.7 MB
Hybrid MobileNetV2	98.38%	96.31%	97.47%	96.88%	11.65	3.01 M	18.7 MB

The hybrid model produced the highest binary test accuracy and the highest photo recall/F1. Relative to pure MobileNetV2 transfer learning, the gain in accuracy was modest (+0.36 percentage points), but photo recall improved from 96.20% to 97.47%, which is important when missed photos are costly.

The scratch CNN remained faster per image in the recorded benchmark but was substantially less accurate and far larger on disk than the transfer-learning models. This made it a weaker practical choice despite its simpler conceptual structure.

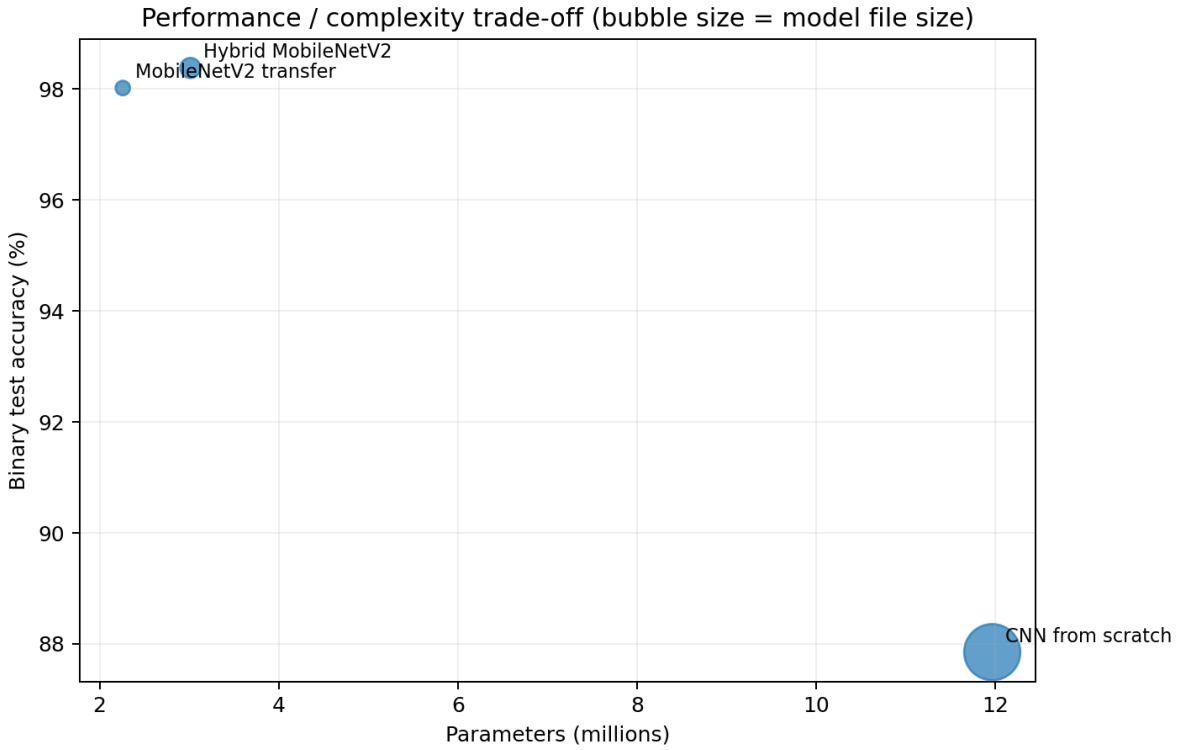
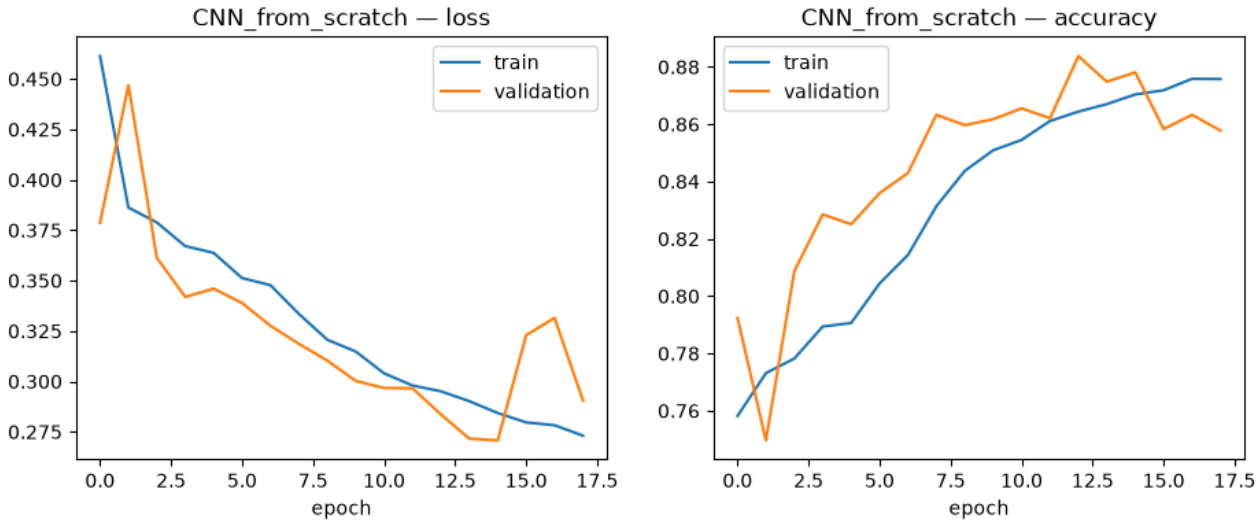


Figure 13. Accuracy/complexity trade-off in the final binary experiment.



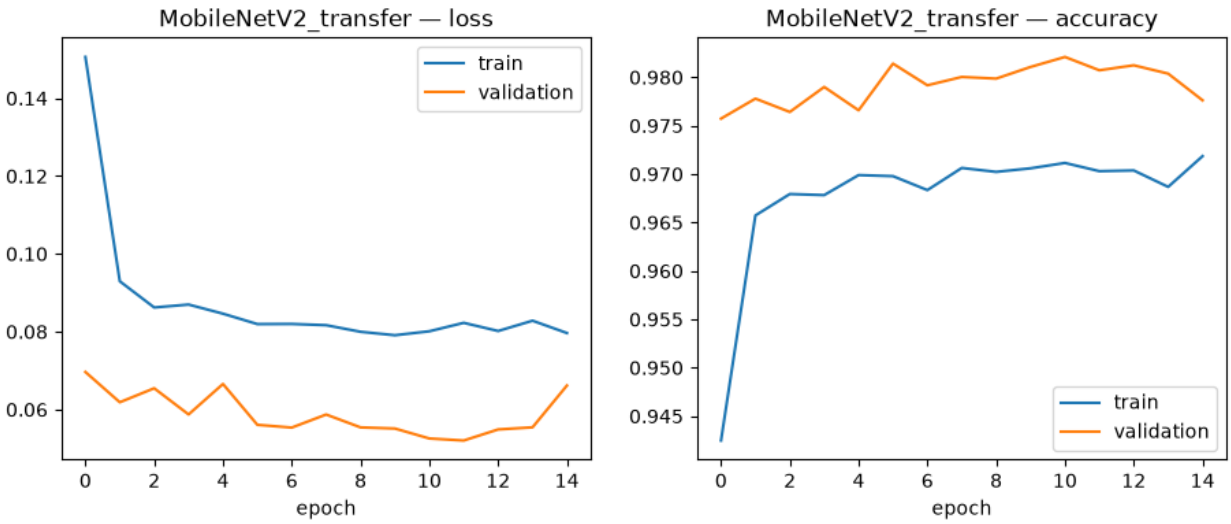


Figure 14. Binary MobileNetV2 transfer-learning training/validation curves.

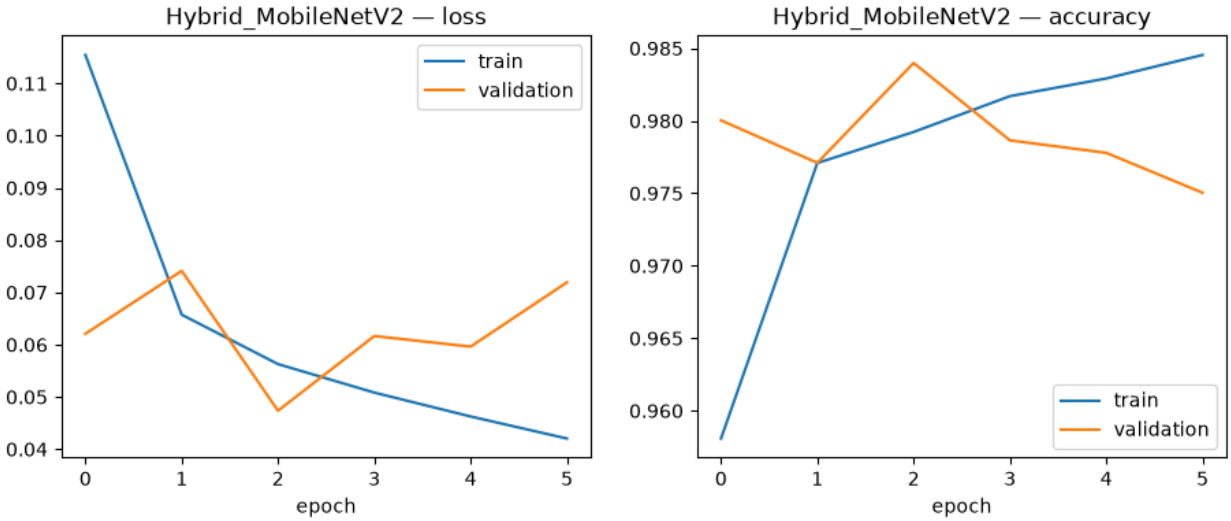


Figure 15. Binary hybrid model training/validation curves.

7. Architecture and Hyperparameter Exploration

During the model selection, we was not limited to changing the model family. The team also varied network depth, hidden-layer width, normalization, dropout, and output structure. The table below summarizes the most important recorded variants.

Family / variant	Recorded architecture	Regularization / notes	Outcome / role
Binary MLP (early)	Flatten → 512 → 256 → 256 → 1	Dropout 0.4 after each dense layer; ~77.3 M parameters	Large baseline; illustrates the cost of flattening high-resolution images.
Multiclass MLP	Flatten → 128 → BN → 64 → 5	Dropout 0.5; BatchNormalization added after unstable earlier version	Stabilized training but only 64.44% test accuracy.
CNN from scratch	Conv32 → Pool → Conv64 → Pool → Dense64	Dropout 0.25 after conv blocks and 0.5 before output	Better inductive bias than MLP; still behind transfer learning.
MobileNetV2 transfer	Frozen MobileNetV2 → GAP → output	Dropout 0.2	Best multiclass result; very compact and fast to train.

Family / variant	Recorded architecture	Regularization / notes	Outcome / role
Hybrid MobileNetV2	Frozen MobileNetV2 → Conv64 → Pool → Dense32 → output	Small custom head; no extra dropout in final recorded head	Best final binary result and retained architecture.
Padding experiment	Conv padding: valid vs same	Same preserves spatial dimensions; valid progressively reduces them	Same reached 78.71% best validation accuracy vs 76.82% for valid, with higher compute cost.

The dropout values illustrate the broader tuning process: 0.2 in pure transfer learning, 0.25 in convolutional blocks, 0.4 in the early binary MLP, and 0.5 around dense layers in the later MLP/CNN baselines. These values were used to control overfitting according to model capacity and where the parameters were concentrated.

Layer width was also progressively reduced as better feature extractors were introduced. The early MLP needed hundreds of fully connected units because it operated directly on 150,528 pixel values. The hybrid model only required Dense(32) because MobileNetV2 had already transformed the image into a compact semantic feature map.

7.1 Convolution Padding: “valid” vs “same”

Padding was also treated as an architectural hyperparameter rather than left at its default value. **Two otherwise comparable convolutional configurations were tested using “valid” and “same” padding.** The recorded experiment shows that “same” padding preserved larger spatial feature maps and produced the higher best validation accuracy, at the cost of additional parameters and longer training.

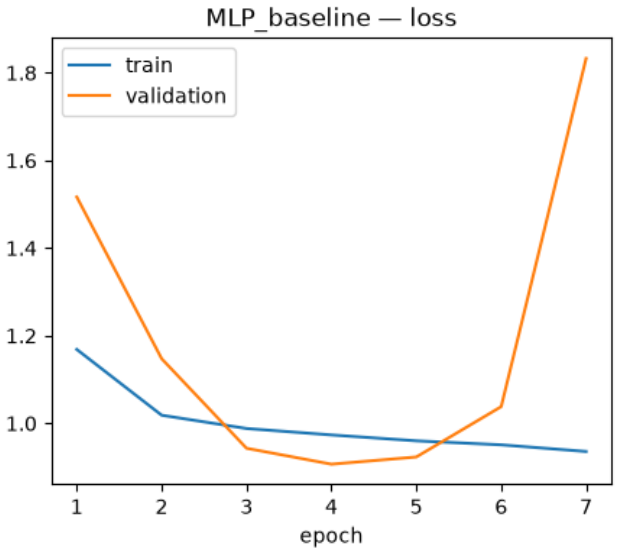
Padding	Conv output sizes	Parameters	Epochs	Best val. accuracy	Training (s)	Seconds / epoch
valid	222×222 → 109×109	11,963,717	4	0.768238	140.8	35.2
same	224×224 → 112×112	12,864,837	5	0.787097	245.5	49.1

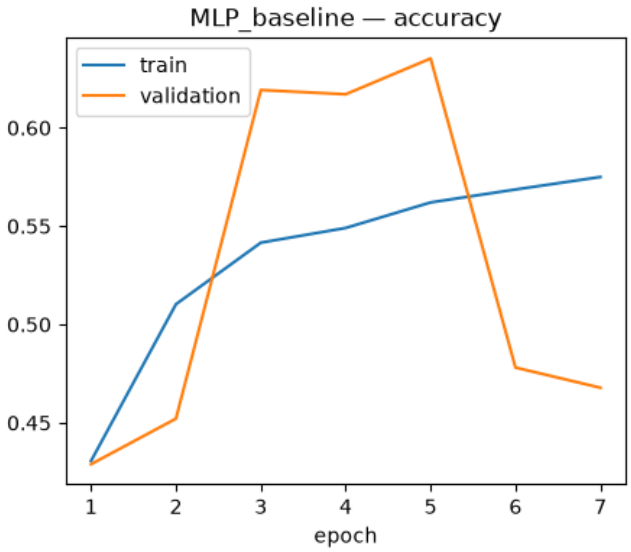
padding	conv output sizes	parameters	epochs	best val accuracy	training seconds	seconds per epoch
valid	222x222 -> 109x109	11963717	4	0.768238	140.8	35.2
same	224x224 -> 112x112	12864837	5	0.787097	245.5	49.1

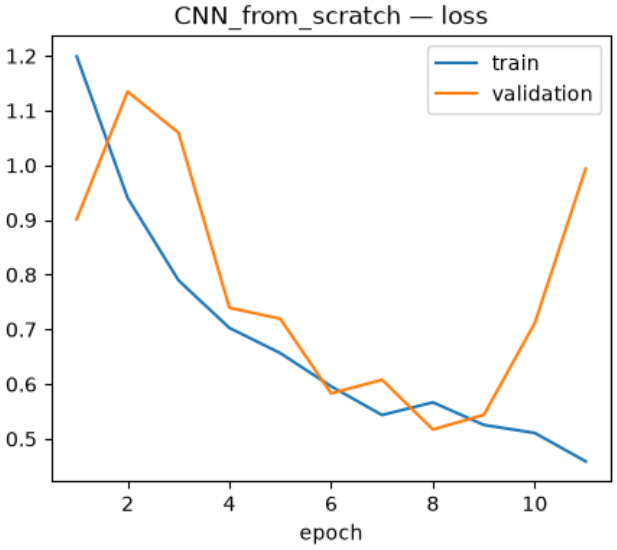
Figure 16. Recorded comparison of “valid” and “same” convolution padding.

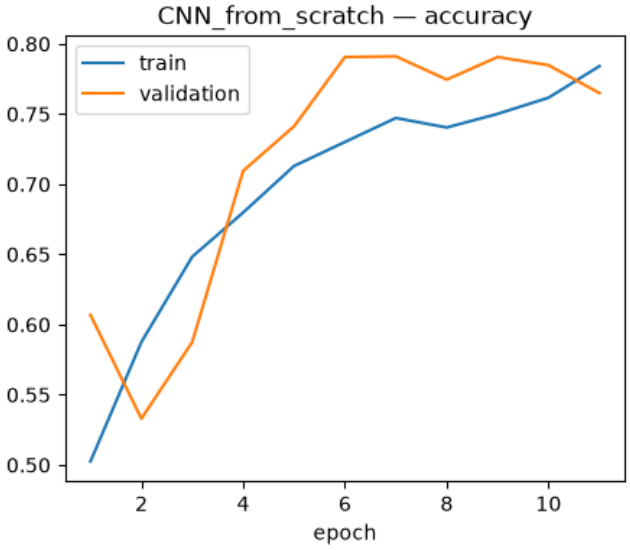
Interpretation. Using “same” padding increased the best validation accuracy from 0.768238 to 0.787097, a gain of 0.018859 (about 1.89 percentage points). This improvement came with 901,120 additional parameters (+7.5%), 13.9 more seconds per epoch (+39.5%), and 104.7 more seconds of total recorded training time (+74.4%). The result illustrates a recurring trade-off in the project: preserving more spatial information can improve predictive performance, but it also increases computation and model complexity.

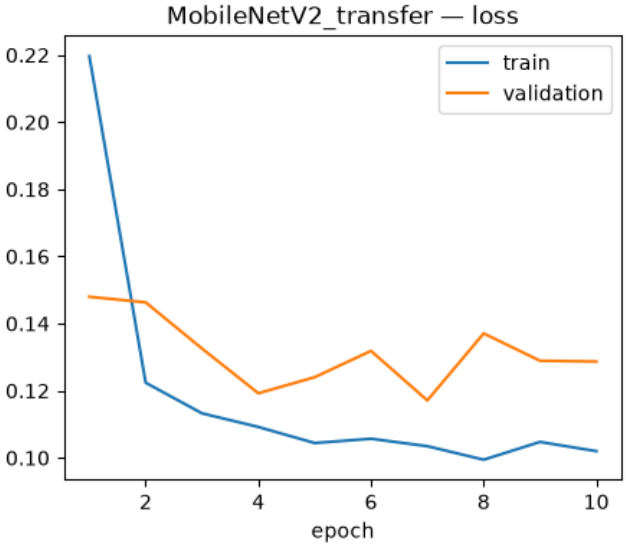
Decision. For this comparison, “same” padding was the stronger accuracy-oriented choice. This result was consistent with the later retained hybrid architecture, whose custom Conv2D head also uses “same” padding. The padding choice was nevertheless evaluated as one component of the full configuration rather than as an isolated criterion.

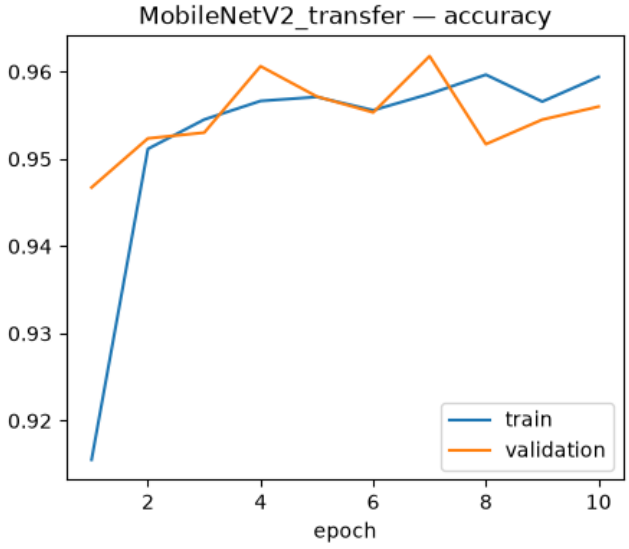

MLP_baseline — loss
train
validation
epoch

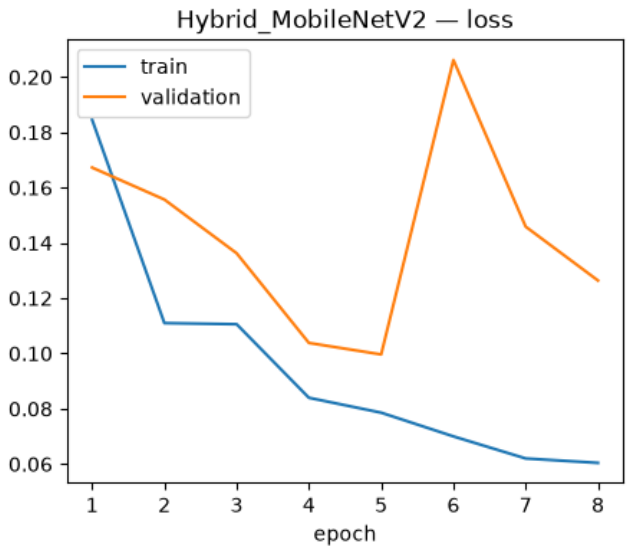

MLP_baseline — accuracy
train
validation
epoch

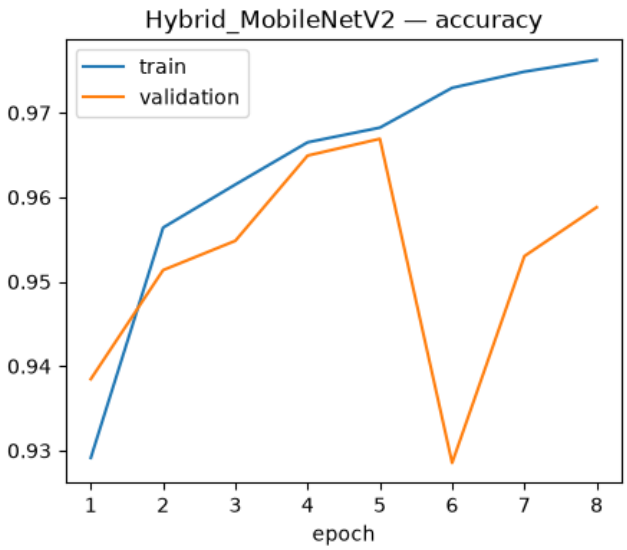

CNN_from_scratch — loss
train
validation
epoch


CNN_from_scratch — accuracy
train
validation
epoch


MobileNetV2_transfer — loss
train
validation
epoch


MobileNetV2_transfer — accuracy
train
validation
epoch





7.2 Second Diagnostic and Improvement Cycle (9 October 2026)

A later five-class diagnostic run was used to understand why the simple models were not only weaker than MobileNetV2, but also much less repeatable. The same model code produced materially different test scores across runs: the MLP moved from 61.3% to 47.3%, while the scratch CNN moved from 76.7% to 67.9%. In contrast, the two MobileNetV2-based approaches remained close to 96%, indicating that pretrained features were substantially more stable.

Model	Observed test accuracy across runs	Diagnostic	Main explanation recorded
MLP	61.3% (7 Oct) → 47.3% (9 Oct)	Unstable + underfitting	Very large dense layer (~19M weights in that version), learning rate too high, and too much simultaneous regularization.
CNN from scratch	76.7% → 67.9%	Overfitting + instability	Only two convolution layers, ~12.8M weights concentrated in Dense, and no BatchNormalization.
MobileNetV2 transfer	96.1% → 96.3%	Bias / plateau	Frozen backbone had reached the limit of the features learned from ImageNet.
Hybrid	96.3% → 96.6%	Mild overfitting	Large custom convolutional head without enough dropout.

This comparison changed the interpretation of the weaker models: the issue was not simply "MLP/CNN are worse," but that their original configurations combined architecture, optimization, and regularization choices that made learning unnecessarily difficult or unstable.

7.3 Six Targeted Changes Introduced

1. More realistic augmentation. The earlier pipeline allowed rotations up to approximately ±36°. The revised augmentation used ±7° rotation, ±5% translation, ±10% zoom, and ±20% contrast. The goal was to reproduce plausible scan/photo variability instead of training on unrealistic orientations.

2. A smaller and more stable MLP. Rather than flattening the full 224×224 image, AveragePooling2D(4) reduces the input to 56×56 before flattening. This cut the MLP from about 19.3M to about 1.2M weights in the documented comparison. The learning rate was reduced from 1e-3 to 1e-4 and augmentation was removed for this baseline.

3. A deeper, better regularized scratch CNN. The two-convolution design was replaced by four blocks containing two convolutions each, with filter widths 32 → 64 → 128 → 256, BatchNormalization, and pooling. GlobalAveragePooling2D replaced Flatten, reducing the dense bottleneck and bringing the network to about 1.18M parameters.

4. A regularized hybrid head. Dropout(0.3) was added before the decision layers to reduce the small train/validation gap observed in the hybrid model.

5. More patient optimization. EarlyStopping was increased from four to eight epochs without improvement, and ReduceLROnPlateau was added to divide the learning rate by three after three stagnant epochs. This allows large optimization steps early in training and smaller steps near a local optimum.

7.4 First Measured Result After the Changes: MLP Stability

At the time of the 9 October diagnostic report, only the revised MLP had completed retraining. Its best validation accuracy increased to 74.3%, compared with a retained validation point around 47.1% in the unstable 9 October run. The improvement appeared immediately after the learning rate was reduced and then remained in the low-to-mid 70% range, supporting the diagnosis that optimization stability was a major problem.

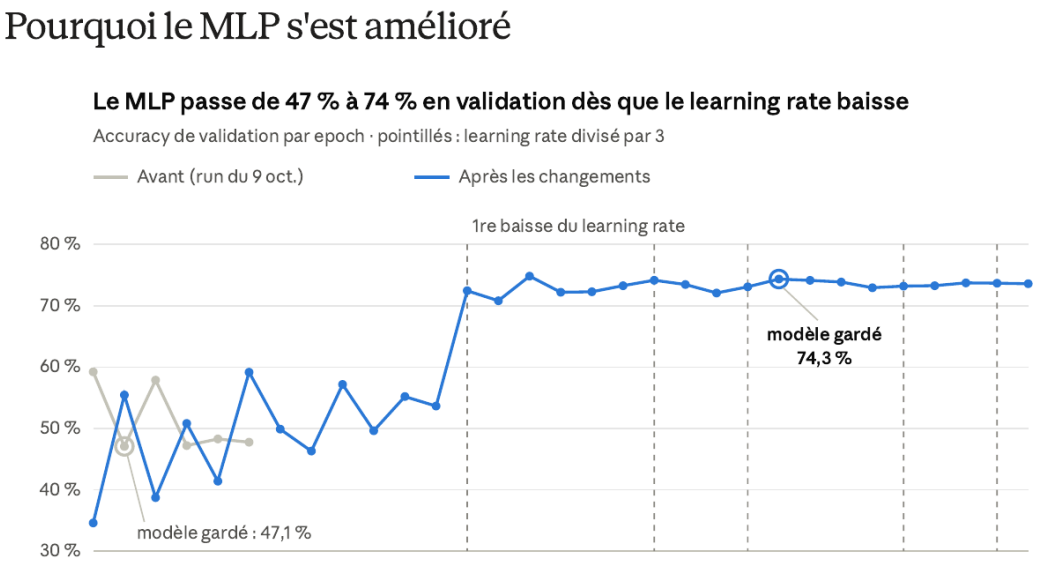


Figure 17. MLP validation accuracy before and after the 9 October changes. The revised run becomes markedly more stable after the first learning-rate reduction and retains a 74.3% validation checkpoint.

> Important interpretation: 74.3% was a validation result, not yet a test result. The source report explicitly marked the revised CNN, transfer, hybrid, and fine-tuned variants as still in progress. Their forecast values therefore remain hypotheses and are not used as measured evidence in the final model comparison.

7.5 GPU Compatibility Issue Found During Retraining

The revised MLP initially failed on GPU because `Resizing(..., interpolation="area")` was incompatible with the XLA-compiled training path; replacing it with `AveragePooling2D(4)` solved the issue and confirmed the need to

test models under the actual GPU execution settings. Additional validation risks identified were the small statistical gap between transfer and hybrid models, possible reliance on aspect ratio as a shortcut, and near-duplicate leakage not detected by MD5 alone.

8. Training Controls and Bias/Variance Analysis

All serious model comparisons used the same general training safeguards: a fixed random seed, shuffled training data, unshuffled validation/test data, EarlyStopping, ModelCheckpoint, and TensorBoard/CSV logging. This ensured that architecture comparisons were not dominated by accidental differences in data order or by continuing training after validation performance had stopped improving. A later improvement cycle deliberately changed two of those controls—more patient early stopping and adaptive learning-rate reduction—to address the instability observed on 9 October.

- EarlyStopping monitored validation loss with patience of three epochs and restored the best weights.
- Later diagnostic cycle: EarlyStopping patience was increased to 8 and ReduceLROnPlateau was added (factor 1/3 after 3 stagnant epochs). This was treated as a new experiment, not silently mixed with the earlier comparisons.
- ModelCheckpoint saved only the best validation-loss model.
- TensorBoard and CSV histories supported comparison of training and validation curves.
- Class weights were computed from the training subset only, preventing information from validation/test data from influencing training.
- Data augmentation was active only during training; validation and test images remained deterministic.

Bias/variance behavior differed strongly by architecture. The MLP and scratch CNN displayed clear capacity or optimization limitations in the multiclass task. Transfer learning reduced bias dramatically because the backbone already contained generic image features. The hybrid binary model converged quickly, but its validation loss began to rise after the best epoch while training loss continued falling; EarlyStopping therefore played an important role in retaining the best generalizing weights rather than the final epoch.

9. Comparative Interpretation and Final Selection

The exploration produced several complementary findings rather than a single “winner” in every setting.

Question	Best-supported conclusion
Binary or multiclass?	Multiclass is valuable for diagnostics, but binary better matches the deliverable and produced the strongest final photo/non-photo performance.
70/15/15 or 80/10/10?	70/15/15 was retained because it gave better observed project results and provides a stronger validation signal.
MLP or CNN?	CNNs are structurally better suited to images; MLPs were parameter-heavy and substantially weaker.
Scratch CNN or transfer learning?	Transfer learning was decisively stronger in both multiclass and binary tests.
Pure transfer or hybrid?	Pure transfer was slightly best for the five-class task; the hybrid was best for the final binary task.
How to control overfitting?	Data augmentation, dropout where appropriate, class weighting, and EarlyStopping/ModelCheckpoint were effective together.

Why the final model was kept

The retained configuration was not chosen because it was the most complex. It was kept because, among the final combinations tested for the actual binary objective, it delivered the best measured balance of accuracy, photo recall/F1, compactness, and controlled validation behavior. In other words, the final pipeline represents the best combination found through the exploration, not an arbitrary architecture choice.

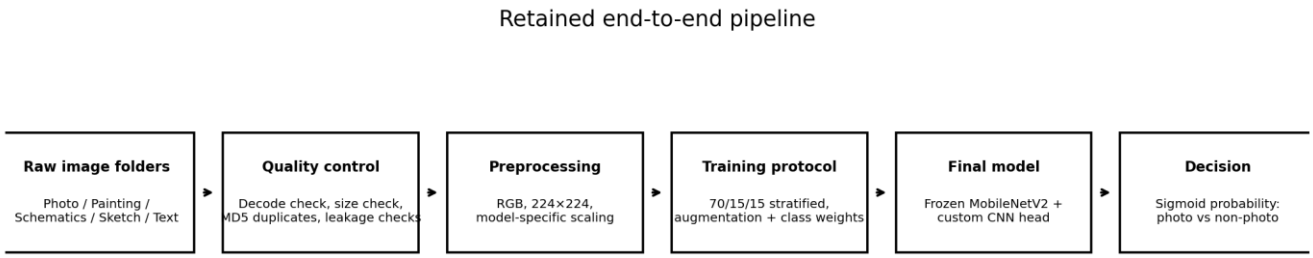


Figure 18. Final retained pipeline after the exploratory process.

9.1 Final retained configuration

Component	Retained choice
Problem formulation	Binary: Photo vs Non-photo
Input	RGB images resized to 224×224
Quality control	TensorFlow readability, minimum size, MD5 duplicates, cross-split leakage checks
Split	70% train / 15% validation / 15% test, stratified by original category
Augmentation	Horizontal flip + small rotation + small zoom
Imbalance handling	Balanced class weights computed on training data
Feature extractor	MobileNetV2 pretrained on ImageNet, frozen
Custom head	Conv2D(64, 3×3, same) → MaxPool(2×2) → Flatten → Dense(32, ReLU) → Dense(1, sigmoid)
Optimizer / loss	Adam (1e-3) + binary cross-entropy
Training controls	EarlyStopping + restore best weights + ModelCheckpoint + TensorBoard
Recorded test result	98.38% accuracy; 96.88% photo F1

Conclusion

The project explored multiple credible paths before converging on the final solution. The team first characterized and cleaned a heterogeneous image dataset, then compared multiclass and binary objectives, experimented with two split ratios, refined preprocessing and leakage checks, and evaluated MLP, CNN, pure transfer-learning, and hybrid architectures with different layer sizes and dropout strategies.

The exploration showed that the biggest gains came from combining a clean and leakage-controlled dataset with pretrained visual representations. Multiclass learning was valuable for understanding errors, especially painting-versus-photo confusion, but the binary formulation was better aligned with the required decision. Among the final binary experiments, the hybrid MobileNetV2 architecture achieved the strongest recorded result: 98.38% test accuracy, 96.31% photo precision, 97.47% photo recall, and 96.88% photo F1.

The retained model and pipeline therefore correspond to the best combination found during the project's exploratory phase. The final choice is the result of comparing problem formulations, data splits, preprocessing strategies, regularization choices, and model architectures—not merely selecting the most sophisticated network. The 9 October diagnostic cycle also documents how the team continued to challenge that choice by diagnosing unstable baselines, testing more realistic augmentation and optimization controls, and defining fine-tuning and robustness checks for subsequent iterations.